LEARN BEING FORWARD

Programming — Java Programming

Building a capstone console app

COMPLETE LEARNING GUIDE

What You'll Learn

This capstone sub-module pulls together everything from core syntax and OOP through collections, exceptions, and file I/O into one coherent, buildable artifact: a complete, menu-driven console application. Rather than introducing new language features, the focus here is the engineering judgment that separates a toy script from a maintainable small program — how to split responsibilities across classes, where to put validation, how to persist state between runs, and how to organize files into packages as a project grows past a single class. The running example throughout is a simple Task Tracker, but every technique (domain modeling, an interactive menu loop, file-based persistence, layered error handling, and packaging into a runnable JAR) transfers directly to any console application you build afterward.

1. Scoping the App and Designing the Domain Model

Before writing any code, a capstone project benefits from five minutes spent naming its core entities and their responsibilities: a Task Tracker needs a Task (the data), a TaskManager (the behavior — add, complete, remove, list), and a thin Main/UI layer (reads input, prints output, delegates everything else). This separation — keeping the 'what the data is' distinct from 'what operations are allowed on it' distinct from 'how the user interacts with it' — is the single-responsibility principle applied practically, and it's what makes a 200-line console app extendable later instead of one tangled main() method that's painful to modify safely.

2. Modeling Domain Objects with Classes and Records

Java 17's record keyword is a perfect fit for simple, immutable data carriers — record Task(int id, String title, boolean done) generates a constructor, getters, equals(), hashCode(), and toString() automatically, eliminating the boilerplate a traditional class would need for pure data. Behavior-heavy, mutable objects like the TaskManager that owns the collection of tasks and coordinates operations on them are better written as a regular class, since records are designed to be immutable value types, not stateful service objects.

```java
public record Task(int id, String title, boolean done) {
    public Task markDone() {
        return new Task(id, title, true); // records are immutable — return a new copy
    }
}
```

3. Building the Interactive Menu Loop with Scanner

The classic console-app shape is a while(true) loop that prints a menu, reads the user's choice with Scanner, dispatches to the matching method via a switch, and loops back — exiting only when the user explicitly selects a quit option. Validating the raw input before trusting it (catching a non-numeric menu choice, re-prompting instead of crashing) belongs in this loop, kept separate from the TaskManager's own business logic, so the core logic stays usable even if the UI layer is later swapped for something other than a console.

```java
Scanner scanner = new Scanner(System.in);
boolean running = true;

while (running) {
    System.out.println("1) Add task  2) List tasks  3) Complete task  4) Quit");
    String choice = scanner.nextLine().trim();

    switch (choice) {
        case "1" -> addTaskFlow(scanner);
        case "2" -> taskManager.listTasks();
        case "3" -> completeTaskFlow(scanner);
        case "4" -> running = false;
        default -> System.out.println("Invalid choice, try again.");
    }
}
```

4. Storing and Retrieving Data with Collections

The TaskManager owns a private backing collection — typically a Map<Integer, Task> keyed by task ID for O(1) lookup, update, and removal by ID, or a simple List<Task> if sequential scanning is acceptable for the app's expected scale — and exposes only well-named methods (addTask, completeTask, removeTask, listTasks) rather than the raw collection itself. Keeping the collection private and mutating it only through these methods means every rule about valid state (an ID must be unique, a completed task can't be 'un-added') is enforced in exactly one place.

```java
public class TaskManager {
    private final Map<Integer, Task> tasks = new LinkedHashMap<>();
    private int nextId = 1;

    public Task addTask(String title) {
        Task task = new Task(nextId++, title, false);
        tasks.put(task.id(), task);
        return task;
    }

    public void completeTask(int id) {
        Task existing = tasks.get(id);
        if (existing == null) throw new NoSuchElementException("No task with id
" + id);        tasks.put(id, existing.markDone());
    }
}
```

5. Persisting State to a File Between Runs

Without persistence, every task entered disappears the moment the program exits — a simple fix is writing each task as one line of a delimited text file (id,title,done) on every change or on exit, and reading that file back into the TaskManager's collection on startup before the menu loop begins. Using java.nio.file.Files with try-with-resources keeps the save/load logic short and leak-free, and isolating this logic into its own method (or even its own small class) keeps the TaskManager's core logic independent of exactly how persistence is implemented.

```java
public void saveToFile(Path path) throws IOException {
    List<String> lines = tasks.values().stream()
        .map(t -> t.id() + "," + t.title() + "," + t.done())
        .toList();
    Files.write(path, lines);
}

public void loadFromFile(Path path) throws IOException {
    if (!Files.exists(path)) return;
    for (String line : Files.readAllLines(path)) {
        String[] parts = line.split(",", 3);
        Task task = new Task(Integer.parseInt(parts[0]), parts[1],
Boolean.parseBoolean(parts[2]));        tasks.put(task.id(), task);
        nextId = Math.max(nextId, task.id() + 1);
    }
}
```

6. Validating Input and Handling Errors Gracefully

User input is the single most unreliable input source in the whole program — a menu choice, a task ID, or a title might be malformed, empty, or simply wrong — so every point where raw text is converted to a number or looked up by ID needs a deliberate catch for NumberFormatException or a custom exception like TaskNotFoundException, with a clear message printed back to the user rather than an uncaught stack trace. The guiding principle: the menu loop should never crash from bad input; at worst, it should print an error and re-prompt.

```java
try {
    int id = Integer.parseInt(scanner.nextLine().trim());
    taskManager.completeTask(id);
    System.out.println("Marked task " + id + " as done.");
} catch (NumberFormatException e) {
    System.out.println("Please enter a valid numeric task ID.");
} catch (NoSuchElementException e) {
    System.out.println(e.getMessage());
}
```

7. Organizing Code into Packages

Once the project grows past a handful of classes, grouping them into packages by role — model (Task), service (TaskManager), and app or ui (Main, the menu loop) — keeps related files together and makes the project's structure legible at a glance, the same way the earlier sub-modules' class designs separated data from behavior from presentation. Each package maps to a real folder on disk matching its dotted name (com.example.tasktracker.model lives in com/example/tasktracker/model/), and Main stays the single, thin entry point that wires the other layers together rather than containing business logic itself.

8. Compiling, Packaging, and Running the Finished App

For a multi-file project, javac compiles every .java file at once (javac com/example/tasktracker/**/*.java, or more simply via a build tool), and java runs it by fully-qualified main class name. To distribute the finished app as a single runnable file, jar cfe app.jar com.example.tasktracker.Main -C out . packages the compiled classes into a JAR with a manifest pointing at the main class, runnable afterward with java -jar app.jar. For anything beyond a single capstone exercise, reaching for a real build tool (Maven or Gradle) to manage compilation, dependencies, and packaging is the practical next step once manual javac/jar commands become tedious.

Common Pitfalls

- Putting all logic directly inside main() instead of separating it into model and service classes, making the program hard to extend or reason about.

- Not validating Scanner input before converting it, letting an uncaught NumberFormatException crash the whole menu loop on one bad keystroke.
- Forgetting to actually call the save-to-file logic before the program exits, silently losing all data entered during that run.
- Hardcoding an absolute, machine-specific file path for persistence instead of a relative path or a path built from a configurable constant.
- Exposing the TaskManager's backing collection directly instead of through methods, letting calling code bypass validation and put it into an inconsistent state.
- Wrapping the entire menu loop in one broad catch (Exception e) instead of handling specific, expected failure cases with clear, distinct messages.
- Forgetting to update the 'next ID' counter correctly after reloading persisted tasks, causing newly added tasks to collide with existing IDs.

Key Takeaways

- Split the project into clear layers — domain data (Task), business logic (TaskManager), and the console UI loop — rather than one large main() method.
- Java 17 records are well suited to small, immutable data carriers; keep mutable, behavior-heavy logic in regular classes.
- Keep a service class's backing collection private and only mutate it through well-named public methods that enforce its own invariants.
- Persist state explicitly with java.nio.file on every meaningful change (or on exit) and reload it on startup, or data vanishes between runs.
- Validate every piece of raw user input at the boundary, and handle expected failure cases (bad ID, missing file) with specific, readable messages instead of a blanket catch.
- Organize files into packages by role (model/service/app) as soon as the project outgrows a single file, and package the finished result into a runnable JAR.

Learn More

Oracle — The Java Tutorials: Packages
https://docs.oracle.com/javase/tutorial/java/package/index.html

Oracle — Record Classes (JEP 395)
https://docs.oracle.com/en/java/javase/17/language/records.html

Baeldung — Creating a Java JAR File with Maven
https://www.baeldung.com/java-create-jar